

C# - PHẦN 8.1: LIST<T>

Collections và Generics - Tài liệu học và ôn tập

Lý thuyết đầy đủ • Ví dụ C# • Phân tích • Lỗi thường gặp • Bài tập tự luyện

Phạm vi tài liệu

Tài liệu tập trung vào List<T> và cách đóng gói collection trong class. Phần LINQ chưa được sử dụng; các thao tác tìm kiếm, kiểm tra trùng và xóa được thực hiện bằng foreach, if và break để người học hiểu rõ cơ chế.

Tài liệu này được thiết kế để bạn đọc lại trước khi làm bài ProductCatalog. Phần cuối chỉ có đề bài và checklist, không kèm đáp án mẫu.

Mục lục nội dung

1. Mục tiêu bài học
2. Collection là gì?
3. List<T> và ý nghĩa của T
4. Khởi tạo danh sách
5. Thêm phần tử: Add và AddRange
6. Count, index và truy cập phần tử
7. Duyệt danh sách bằng foreach và for
8. Xóa phần tử: Remove, RemoveAt, Clear
9. Contains và vấn đề so sánh object
10. Tìm kiếm thủ công bằng foreach
11. Cập nhật object trong List
12. Encapsulation với collection
13. readonly và chia sẻ reference
14. Sao chép danh sách và shallow copy
15. Ví dụ hoàn chỉnh Course - Student
16. Lỗi thường gặp
17. Tóm tắt cần nhớ
18. Câu hỏi tự kiểm tra
19. Bài tập ProductCatalog (không có đáp án)

Cách học khuyên dùng

Đọc từng mục, tự chạy lại code, sau đó đóng tài liệu và giải thích lại bằng lời. Chỉ chuyển sang bài tập khi bạn hiểu được vì sao public List<T> vẫn có thể bị sửa dù property dùng private set.

1. Mục tiêu bài học

Sau khi hoàn thành phần này, bạn cần có thể:

- Giải thích collection và List<T> bằng lời của mình.
- Hiểu T đại diện cho kiểu phần tử của danh sách.
- Khởi tạo danh sách rỗng hoặc có dữ liệu ban đầu.
- Sử dụng Add, AddRange, Count, index, foreach và for.
- Xóa phần tử bằng Remove, RemoveAt và Clear.
- Hiểu giới hạn của Contains khi làm việc với object tự định nghĩa.
- Tìm kiếm và cập nhật object bằng vòng lặp, chưa cần LINQ.
- Đóng gói collection bằng private readonly List<T> và các method quản lý.
- Giải thích reference sharing và shallow copy ở mức cơ bản.
- Tự xây dựng ProductCatalog quản lý nhiều Product.

Trọng tâm

Phần khó nhất không phải nhớ tên method của List<T>, mà là hiểu cách bảo vệ danh sách bên trong class và kiểm soát mọi thay đổi thông qua method nghiệp vụ.

2. Collection là gì?

Collection là cấu trúc dùng để lưu trữ và quản lý nhiều phần tử. Khi chỉ có một object, ta có thể dùng một biến đơn. Khi số lượng object tăng lên, việc tạo nhiều biến rồi rạc trở nên khó quản lý.

Ba biến riêng lẻ

```
Student student1 = new Student("SV01", "An");
Student student2 = new Student("SV02", "Binh");
Student student3 = new Student("SV03", "Cuong");
```

Cách trên vẫn chạy được, nhưng khó duyệt, tìm kiếm, xóa hoặc đếm. Ta thay bằng collection:

```
List<Student> students = new List<Student>();

students.Add(student1);
students.Add(student2);
students.Add(student3);
```

Ghi nhớ

Collection giúp gom nhiều phần tử vào một cấu trúc có thể duyệt và quản lý thống nhất.

3. List<T> và ý nghĩa của T

List<T> là một collection có thứ tự, cho phép lưu nhiều phần tử cùng một kiểu dữ liệu. Ký hiệu T là tham số kiểu (type parameter). Khi sử dụng List<T>, ta thay T bằng kiểu dữ liệu cụ thể.

```
List<int> numbers = new List<int>();
List<string> names = new List<string>();
List<Student> students = new List<Student>();
```

Khai báo	Ý nghĩa
List<int>	Danh sách chỉ chứa số nguyên.
List<string>	Danh sách chỉ chứa chuỗi.
List<Student>	Danh sách chỉ chứa object Student.

Nhờ chỉ rõ kiểu phần tử, trình biên dịch có thể ngăn dữ liệu sai kiểu ngay khi viết code:

```
List<Student> students = new List<Student>();

students.Add(new Student("SV01", "An")); // Hợp lệ
students.Add("Binh");                    // Lỗi biên dịch
students.Add(10);                        // Lỗi biên dịch
```

T là gì?

T không phải một kiểu dữ liệu cố định. Nó là vị trí đại diện cho kiểu sẽ được cung cấp khi sử dụng generic. Trong List<Student>, T được thay bằng Student.

4. Khởi tạo danh sách

4.1. Danh sách rỗng

```
List<Student> students = new List<Student>();
```

Ngay sau khi tạo, danh sách tồn tại nhưng chưa có phần tử:

```
Console.WriteLine(students.Count); // 0
```

4.2. Danh sách có dữ liệu ban đầu

```
List<Student> students = new List<Student>
{
    new Student("SV01", "An"),
    new Student("SV02", "Binh"),
    new Student("SV03", "Cuong")
};
```

4.3. Target-typed new

C# hiện đại cho phép lược bỏ kiểu ở vế phải khi trình biên dịch đã biết kiểu từ vế trái:

```
List<Student> students = new()
{
    new Student("SV01", "An"),
    new Student("SV02", "Binh")
};
```

Khuyến nghị khi mới học

Bạn có thể tiếp tục viết new List<Student>() đầy đủ. Khi đã quen, hãy dùng new() để code gọn hơn.

5. Thêm phần tử: Add và AddRange

5.1. Add() - thêm một phần tử

```
Student student = new Student("SV04", "Dung");
students.Add(student);
```

Hoặc tạo object trực tiếp khi thêm:

```
students.Add(new Student("SV05", "Hoa"));
```

5.2. AddRange() - thêm nhiều phần tử

```
List<Student> newStudents = new List<Student>
{
    new Student("SV06", "Khanh"),
    new Student("SV07", "Lan")
};
```

```
};
students.AddRange(newStudents);
```

Method	Công dụng
Add(element)	Thêm một phần tử.
AddRange(collection)	Thêm toàn bộ phần tử từ collection khác.

Lưu ý

List<T> cho phép phần tử trùng nhau. Việc không cho trùng Id là quy tắc nghiệp vụ do class quản lý danh sách phải tự kiểm tra.

6. Count, index và truy cập phần tử

6.1. Count

Count là số phần tử hiện có trong List:

```
Console.WriteLine(students.Count);
```

6.2. Index bắt đầu từ 0

```
Student firstStudent = students[0];
Console.WriteLine(firstStudent.FullName);
```

Index	Phần tử
0	Phần tử thứ nhất
1	Phần tử thứ hai
2	Phần tử thứ ba

Nếu truy cập index không tồn tại, chương trình ném `ArgumentOutOfRangeException`.

```
if (index >= 0 && index < students.Count)
{
    Console.WriteLine(students[index].FullName);
}
```

Sai lầm phổ biến

Nhầm rằng phần tử đầu tiên có index 1. Trong List<T>, index luôn bắt đầu từ 0.

7. Duyệt danh sách bằng foreach và for

7.1. foreach

```
foreach (Student student in students)
{
    student.DisplayInfo();
}
```

foreach phù hợp khi cần đọc hoặc gọi method cho từng phần tử mà không cần biết index.

7.2. for

```
for (int i = 0; i < students.Count; i++)
{
```

```

        Console.WriteLine($"Index: {i}");
        students[i].DisplayInfo();
    }

```

foreach	for
Dễ đọc, không cần index.	Có index để xử lý theo vị trí.
Phù hợp duyệt tuần tự.	Phù hợp khi cần thay thế/xóa theo vị trí.
Không nên thay đổi cấu trúc list trong lúc duyệt.	Có thể kiểm soát index nhưng vẫn phải cẩn thận khi xóa.

8. Xóa phần tử: Remove, RemoveAt, Clear

8.1. Remove(object)

```

bool removed = students.Remove(student);

if (removed)
{
    Console.WriteLine("Xoa thanh cong");
}
else
{
    Console.WriteLine("Khong tim thay object can xoa");
}

```

Remove trả về true nếu tìm thấy và xóa thành công; ngược lại trả về false.

8.2. RemoveAt(index)

```

if (students.Count > 0)
{
    students.RemoveAt(0);
}

```

8.3. Clear()

```

students.Clear();
Console.WriteLine(students.Count); // 0

```

Clear không làm List thành null

Sau Clear(), object List vẫn tồn tại và có thể Add() phần tử mới. Chỉ nội dung bên trong bị xóa.

9. Contains và vấn đề so sánh object

Contains kiểm tra xem phần tử có trong danh sách hay không:

```

if (students.Contains(student))
{
    Console.WriteLine("Da co trong danh sach");
}

```

Với object tự định nghĩa, Contains mặc định thường so sánh theo reference nếu class chưa tùy chỉnh quy tắc equality.

```

Student s1 = new Student("SV01", "An");
Student s2 = new Student("SV01", "An");

students.Add(s1);

Console.WriteLine(students.Contains(s1)); // true
Console.WriteLine(students.Contains(s2)); // thường false

```

Vì sao s2 thường là false?

Dù dữ liệu giống nhau, s1 và s2 là hai object được tạo bởi hai lần new khác nhau. Chúng có hai reference khác nhau.

Trong bài hiện tại, để kiểm tra trùng Id, ta chủ động duyệt foreach và so sánh thuộc tính Id.

10. Tìm kiếm thủ công bằng foreach

```
Student? foundStudent = null;

foreach (Student student in students)
{
    if (student.Id == "SV02")
    {
        foundStudent = student;
        break;
    }
}

if (foundStudent != null)
{
    foundStudent.DisplayInfo();
}
else
{
    Console.WriteLine("Không tìm thấy sinh viên");
}
```

Dấu ? trong Student? cho biết biến có thể giữ object Student hoặc null.

Vai trò của break

Khi đã tìm thấy phần tử cần thiết, break kết thúc vòng lặp ngay, tránh duyệt các phần tử còn lại không cần thiết.

Khi học LINQ, đoạn này có thể viết bằng FirstOrDefault(). Tuy nhiên, hiểu vòng lặp trước giúp bạn biết LINQ đang làm gì.

11. Cập nhật object trong List

List giữ reference tới object. Vì vậy ta có thể tìm object và gọi method để cập nhật trạng thái của chính object đó.

Method thuộc class Student

```
public void ChangeFullName(string newFullName)
{
    if (string.IsNullOrWhiteSpace(newFullName))
    {
        throw new ArgumentException(
            "Tên mới không hợp lệ",
            nameof(newFullName));
    }

    FullName = newFullName.Trim();
}
```

```
foreach (Student student in students)
{
    if (student.Id == "SV02")
    {
        student.ChangeFullName("Tran Van Binh");
        break;
    }
}
```

Liên hệ Encapsulation

Không nên mở public set chỉ để cập nhật object trong List. Hãy giữ private set và cung cấp method có validation.

12. Encapsulation với collection

12.1. Vấn đề của public List<T>

```
public List<Student> Students { get; private set; }
```

private set chỉ ngăn bên ngoài thay thế toàn bộ property:

```
course.Students = new List<Student>(); // Không được
```

Nhưng bên ngoài vẫn có thể sửa nội dung của object List:

```
course.Students.Add(new Student("SV99", "ABC"));
course.Students.Clear();
```

Điểm rất quan trọng

private set bảo vệ việc gán property, nhưng không biến List thành bất biến. Nội dung List vẫn thay đổi được nếu bên ngoài truy cập được reference.

12.2. Dùng field private và method nghiệp vụ

```
public class Course
{
    private readonly List<Student> _students;

    public Course()
    {
        _students = new List<Student>();
    }

    public void AddStudent(Student student)
    {
        if (student == null)
        {
            throw new ArgumentNullException(nameof(student));
        }

        _students.Add(student);
    }
}
```

Bây giờ code bên ngoài không thể truy cập trực tiếp _students. Mọi thay đổi phải đi qua method do Course cung cấp.

13. readonly và chia sẻ reference

13.1. readonly bảo vệ điều gì?

```
private readonly List<Student> _students;
```

readonly ngăn field bị gán sang một object List khác sau constructor:

```
_students = new List<Student>(); // Lỗi nếu thực hiện ngoài constructor
```

Nhưng vẫn có thể thay đổi nội dung của List:

```
_students.Add(student);
_students.Remove(student);
_students.Clear();
```


Câu nhớ

readonly List<T> = không đổi reference sang List khác, nhưng vẫn thay đổi phần tử bên trong List hiện tại.

13.2. Chia sẻ cùng một List

```
List<Student> students = new List<Student>();
Course course = new Course(students);

students.Clear();
```

Nếu constructor chỉ gán `_students = students`, biến `students` bên ngoài và `_students` bên trong cùng trỏ tới một List. Khi bên ngoài `Clear()`, `Course` cũng thấy danh sách bị xóa.

```
public Course(List<Student> students)
{
    _students = students;
}
```

Mô hình reference

`students` bên ngoài và `Course._students` có thể là hai biến khác nhau nhưng cùng tham chiếu tới một object List duy nhất.

14. Sao chép danh sách và shallow copy

Để giảm việc bên ngoài thay đổi trực tiếp danh sách bên trong, constructor có thể tạo một List mới:

```
public Course(List<Student> students)
{
    if (students == null)
    {
        throw new ArgumentNullException(nameof(students));
    }

    _students = new List<Student>(students);
}
```

Bây giờ `Course` có object List riêng. Nếu bên ngoài gọi `students.Clear()`, `_students` không bị xóa theo.

Shallow copy

List mới được tạo, nhưng các object `Student` bên trong vẫn là cùng reference. Nếu một `Student` được thay đổi, cả hai danh sách đều nhìn thấy trạng thái mới của `Student` đó.

```
Student s1 = new Student("SV01", "An");
List<Student> original = new List<Student> { s1 };
List<Student> copy = new List<Student>(original);

s1.ChangeFullName("Nguyen Van An");
// original[0] và copy[0] đều thấy tên mới
```

15. Ví dụ hoàn chỉnh: Course - Student

Ví dụ sau tổng hợp validation, private readonly List, kiểm tra trùng Id, tìm kiếm, xóa và hiển thị. Chưa sử dụng LINQ.

```
public class Student
{
    public string Id { get; }
    public string FullName { get; private set; }
```

```

public Student(string id, string fullName)
{
    if (string.IsNullOrEmpty(id))
        throw new ArgumentException("Id không hợp lệ", nameof(id));

    if (string.IsNullOrEmpty(fullName))
        throw new ArgumentException("Tên không hợp lệ", nameof(fullName));

    Id = id.Trim();
    FullName = fullName.Trim();
}

public void ChangeFullName(string newFullName)
{
    if (string.IsNullOrEmpty(newFullName))
        throw new ArgumentException("Tên mới không hợp lệ", nameof(newFullName));

    FullName = newFullName.Trim();
}

public override string ToString()
{
    return $"Id: {Id}, Name: {FullName}";
}
}

```

```

public class Course
{
    public string Name { get; }
    private readonly List<Student> _students;

    public Course(string name)
    {
        if (string.IsNullOrEmpty(name))
            throw new ArgumentException("Tên khoa học không hợp lệ", nameof(name));

        Name = name.Trim();
        _students = new List<Student>();
    }

    public void AddStudent(Student student)
    {
        if (student == null)
            throw new ArgumentNullException(nameof(student));

        foreach (Student existingStudent in _students)
        {
            if (existingStudent.Id == student.Id)
                throw new InvalidOperationException(
                    $"Sinh viên có Id {student.Id} đã tồn tại");
        }

        _students.Add(student);
    }

    public Student? FindStudentById(string id)
    {
        if (string.IsNullOrEmpty(id))
            throw new ArgumentException("Id không hợp lệ", nameof(id));

        string normalizedId = id.Trim();

        foreach (Student student in _students)
        {
            if (student.Id == normalizedId)
                return student;
        }

        return null;
    }

    public bool RemoveStudentById(string id)
    {
        Student? foundStudent = FindStudentById(id);

        if (foundStudent == null)
            return false;

        return _students.Remove(foundStudent);
    }
}

```

```

    }

    public void DisplayStudents()
    {
        Console.WriteLine($"Course: {Name}");

        if (_students.Count == 0)
        {
            Console.WriteLine("Danh sách sinh viên rỗng");
            return;
        }

        foreach (Student student in _students)
            Console.WriteLine(student);
    }
}

```

Trong `RemoveStudentById()`, ta tái sử dụng `FindStudentById()` thay vì lặp lại toàn bộ logic tìm kiếm. Đây là một bước cải thiện nhỏ về tổ chức code.

16. Lỗi thường gặp

Lỗi	Cách sửa
Nhầm index bắt đầu từ 1.	List bắt đầu từ index 0.
Truy cập <code>students[index]</code> không kiểm tra.	Đảm bảo $0 \leq \text{index} < \text{Count}$.
Cho rằng private set bảo vệ nội dung List.	Dùng field private và method quản lý.
Cho rằng readonly làm List bất biến.	readonly chỉ khóa việc gán reference.
Dùng <code>Contains</code> để kiểm tra trùng Id.	Duyệt và so sánh thuộc tính Id.
Xóa trong foreach đang duyệt cùng List.	Tìm object trước, sau đó <code>Remove</code> ; hoặc dùng chiến lược phù hợp.
Không kiểm tra null khi Add object.	Ném <code>ArgumentNullException</code> .
Không kiểm tra danh sách null trong constructor.	Validation trước khi gán hoặc copy.
Cho rằng <code>new List<Student>(students)</code> sao chép sâu.	Đó chỉ là shallow copy của collection.
Mở public set để cập nhật object.	Dùng method có validation trong chính object.

Lỗi nghiêm trọng cần tránh

Không thay đổi cấu trúc của List bằng `Add/Remove/Clear` ngay trong foreach đang duyệt chính List đó. Điều này thường gây `InvalidOperationException`.

17. Tóm tắt cần nhớ

List<T>

Collection có thứ tự, chứa nhiều phần tử cùng kiểu T và truy cập được bằng index bắt đầu từ 0.

Thành phần	Ý nghĩa
Add	Thêm một phần tử.
AddRange	Thêm nhiều phần tử.
Count	Số lượng phần tử.
Remove	Xóa theo object, trả về bool.
RemoveAt	Xóa theo index.
Clear	Xóa toàn bộ phần tử.
Contains	Kiểm tra phần tử theo quy tắc equality.

Thành phần	Ý nghĩa
foreach	Duyệt tuần tự.
for	Duyệt có index.

Encapsulation collection

Ưu tiên private readonly List<T> và cung cấp Add/Remove/Find/Display method. Đừng để code bên ngoài tùy ý Clear hoặc Add nếu nghiệp vụ cần kiểm soát.

Reference

Hai biến có thể cùng trỏ tới một List. Copy bằng new List<T>(source) tạo List mới nhưng vẫn giữ cùng reference của các object bên trong.

18. Câu hỏi tự kiểm tra

1. Collection giải quyết vấn đề gì so với việc tạo nhiều biến rời rạc?
2. T trong List<T> đại diện cho điều gì?
3. List<Student> có thể Add một string không? Vì sao?
4. Count khác index như thế nào?
5. Tại sao phải kiểm tra index trước khi dùng students[index]?
6. Add và AddRange khác nhau ở đâu?
7. Remove trả về kiểu gì và ý nghĩa của kết quả đó?
8. Clear có làm List trở thành null không?
9. Vì sao Contains(s2) có thể false dù s1 và s2 có cùng dữ liệu?
10. Khi nào nên dùng foreach, khi nào nên dùng for?
11. Tại sao public List<Student> Students { get; private set; } vẫn chưa bảo vệ tốt danh sách?
12. readonly bảo vệ reference hay nội dung List?
13. Điều gì xảy ra nếu Course._students = students và bên ngoài gọi students.Clear()?
14. new List<Student>(students) là shallow copy hay deep copy?
15. Tại sao cập nhật FullName nên thông qua ChangeFullName() thay vì mở public set?

Cách tự đánh giá

Nếu bạn có thể trả lời thành lời và viết được ví dụ nhỏ cho ít nhất 12/15 câu, bạn đã sẵn sàng làm bài ProductCatalog.

19. Bài tập thực hành: ProductCatalog

Không có đáp án trong tài liệu này

Hãy tự làm bằng foreach, if, break và List<T>. Chưa sử dụng LINQ.

19.1. Class Product

- Property Id kiểu string, không được rỗng.
- Property Name kiểu string, không được rỗng.
- Property Price kiểu decimal, phải lớn hơn 0.

- Dùng get-only hoặc private set phù hợp.
- Gọi Trim() cho chuỗi.
- Override ToString() để hiển thị thông tin sản phẩm.

19.2. Class ProductCatalog

- Property Name.
- Field private readonly List<Product> _products.
- Constructor validation tên catalog và khởi tạo danh sách rỗng.
- Không cho code bên ngoài truy cập trực tiếp field _products.

19.3. Các method bắt buộc

```
public void AddProduct(Product product)
```

- product không được null.
- Không được trùng Id.
- Nếu trùng, throw InvalidOperationException.

```
public Product? FindProductById(string id)
```

- id không được rỗng.
- Duyệt foreach.
- Tìm thấy thì return Product.
- Không thấy thì return null.

```
public bool RemoveProductById(string id)
```

- Tìm sản phẩm theo Id.
- Không thấy thì return false.
- Tìm thấy thì xóa và return true.

```
public void DisplayProducts()
```

- In tên catalog.
- Nếu danh sách rỗng thì thông báo.
- Nếu có dữ liệu thì foreach và in từng Product.

19.4. Dữ liệu kiểm thử gợi ý

```
ProductCatalog catalog = new ProductCatalog("Electronics");

catalog.AddProduct(new Product("P01", "Laptop", 20_000_000m));
catalog.AddProduct(new Product("P02", "Mouse", 500_000m));
catalog.AddProduct(new Product("P03", "Keyboard", 1_200_000m));

catalog.DisplayProducts();

Product? found = catalog.FindProductById("P02");

if (found != null)
{
    Console.WriteLine($"Tim thay: {found}");
}

bool removed = catalog.RemoveProductById("P01");
Console.WriteLine(removed ? "Xoa thanh cong" : "Khong tim thay");

catalog.DisplayProducts();
```

19.5. Các trường hợp phải tự kiểm thử thêm

- Thêm product null.
- Thêm hai Product có cùng Id.

- Tìm Id không tồn tại.
- Xóa Id không tồn tại.
- Tạo Product có Price bằng 0 hoặc âm.
- Tạo Product có Id/Name chỉ gồm khoảng trắng.
- Hiển thị catalog rỗng.

19.6. Checklist trước khi gửi bài

- ☐ Product validation đúng.
- ☐ ProductCatalog dùng private readonly List<Product>.
- ☐ Không public trực tiếp danh sách.
- ☐ AddProduct kiểm tra null và Id trùng.
- ☐ FindProductById dùng foreach, không dùng LINQ.
- ☐ RemoveProductById trả về bool đúng nghĩa.
- ☐ DisplayProducts xử lý danh sách rỗng.
- ☐ Tên biến local dùng camelCase.
- ☐ Property/class/method dùng PascalCase.
- ☐ Đã chạy toàn bộ trường hợp kiểm thử.

Hoàn thành Phần 8.1 khi nào?

Bạn hoàn thành phần này khi có thể tự viết ProductCatalog, giải thích được reference sharing, và biết vì sao field private readonly List<T> an toàn hơn public List<T>.

Phần tiếp theo sau khi bài ProductCatalog đạt yêu cầu: Dictionary<TKey, TValue>, HashSet<T>, Queue<T>, Stack<T>, sau đó mới đi sâu vào Generic class/method và LINQ.